CENTRO UNIVERSITÁRIO FILADÉLFIA - UNIFIL

CURSO DE ENGENHARIA DE SOFTWARE

DESENVOLVIMENTO DE PROJETOS II

GABRIEL DINIZ

GABRIEL SATIN

PETAGENDA: SISTEMA WEB DE AGENDAMENTO PARA PET SHOP

Parte I - Documentação e modelagem do sistema

LONDRINA

2026

SUMÁRIO

1 INTRODUÇÃO 3
1.1 Tecnologias utilizadas 3
2 DESENVOLVIMENTO 4
2.1 Requisitos funcionais 4
2.2 Requisitos não funcionais 4
2.3 Diagrama de casos de uso 5
2.4 Diagrama Entidade-Relacionamento (DER) 6
2.5 Diagrama de atividade 7
2.6 Diagrama de classes 8
2.7 Diagramas de sequência 8
2.7.1 UC01 - Manter Clientes 8
2.7.2 UC02 - Manter Pets 9
2.7.3 UC03 - Manter Serviços 11
2.7.4 UC04 - Agendar Atendimento 12
2.7.5 UC05 - Consultar Agenda do Dia 12
3 CONCLUSÃO 13
REFERÊNCIAS 14

1 INTRODUÇÃO

Pet shops de bairro costumam marcar banho, tosa e consultas pelo WhatsApp ou numa agenda de papel. Isso funciona enquanto o movimento é pequeno, mas logo aparecem problemas: dois pets marcados no mesmo horário, cliente que não aparece porque ninguém confirmou, e nenhuma ideia de quanto foi faturado no mês.

O PetAgenda é um sistema web simples para resolver isso. Nele o atendente cadastra os clientes (tutores), os pets de cada cliente e os serviços oferecidos, e faz os agendamentos. O sistema não deixa marcar dois atendimentos no mesmo horário, nem no passado, nem fora do horário de funcionamento. Na tela inicial aparece a agenda do dia, com a opção de marcar o atendimento como concluído.

Este documento corresponde à Parte I do projeto, com o levantamento de requisitos e a modelagem do sistema por meio de diagramas UML e do Diagrama Entidade-Relacionamento.

O trabalho foi desenvolvido em dupla pelos alunos Gabriel Diniz e Gabriel Satin. O código-fonte está no repositório público https://github.com/Gabriel-diniz007/petagenda.

1.1 Tecnologias utilizadas

Quadro 1 - Stack do projeto

Tecnologia	Uso no projeto
Node.js 24	Ambiente de execução do JavaScript no servidor
Express 4	Framework web: rotas, formulários e arquivos estáticos
EJS	Templates HTML renderizados no servidor
SQLite (node:sqlite)	Banco de dados relacional em arquivo, usando o módulo que já vem no Node
HTML5 e CSS3	Interface do usuário
node:test	Testes automatizados
Git e GitHub	Versionamento e trabalho em dupla

Fonte: os autores (2026).

Escolhemos o Node.js porque é uma tecnologia muito usada no mercado e permite usar JavaScript tanto no front quanto no back-end. O SQLite foi escolhido por

não precisar de servidor de banco instalado: o banco é um arquivo criado automaticamente na primeira execução. A partir do Node 22 existe o módulo node:sqlite, então nem foi preciso instalar driver de banco.

2 DESENVOLVIMENTO

Nesta seção estão os requisitos levantados e a modelagem feita antes da implementação.

2.1 Requisitos funcionais

Quadro 2 - Requisitos funcionais

Código	Descrição
RF01	Cadastrar, listar, buscar (por nome ou telefone), editar e excluir clientes (tutores).
RF02	Cadastrar, listar, editar e excluir pets, vinculando cada pet a um cliente.
RF03	Cadastrar, listar, editar e excluir os serviços oferecidos, com preço e duração.
RF04	Agendar atendimentos informando pet, serviço, data e hora.
RF05	Editar e excluir agendamentos, e filtrar a lista por data e por status.
RF06	Marcar um agendamento como concluído ou cancelado.
RF07	Exibir na tela inicial a agenda do dia, o total de clientes, de pets, de agendamentos em aberto e o faturamento do mês (atendimentos concluídos).
RF08	Impedir dois agendamentos em aberto no mesmo dia e horário.
RF09	Impedir agendamento em data/hora que já passou e fora do horário de funcionamento (08h às 18h).
RF10	Impedir a exclusão de cliente com pets, de pet com agendamento em aberto e de serviço já utilizado.
RF11	Mostrar a ficha do cliente com a lista dos seus pets e a idade de cada um.

Fonte: os autores (2026).

2.2 Requisitos não funcionais

Quadro 3 - Requisitos não funcionais

Código	Descrição
RNF01	Aplicação web, acessada pelo navegador.
RNF02	Back-end em JavaScript com Node.js (versão 22.13 ou superior) e framework Express.
RNF03	Dados armazenados em banco relacional SQLite, usando o módulo nativo node:sqlite.

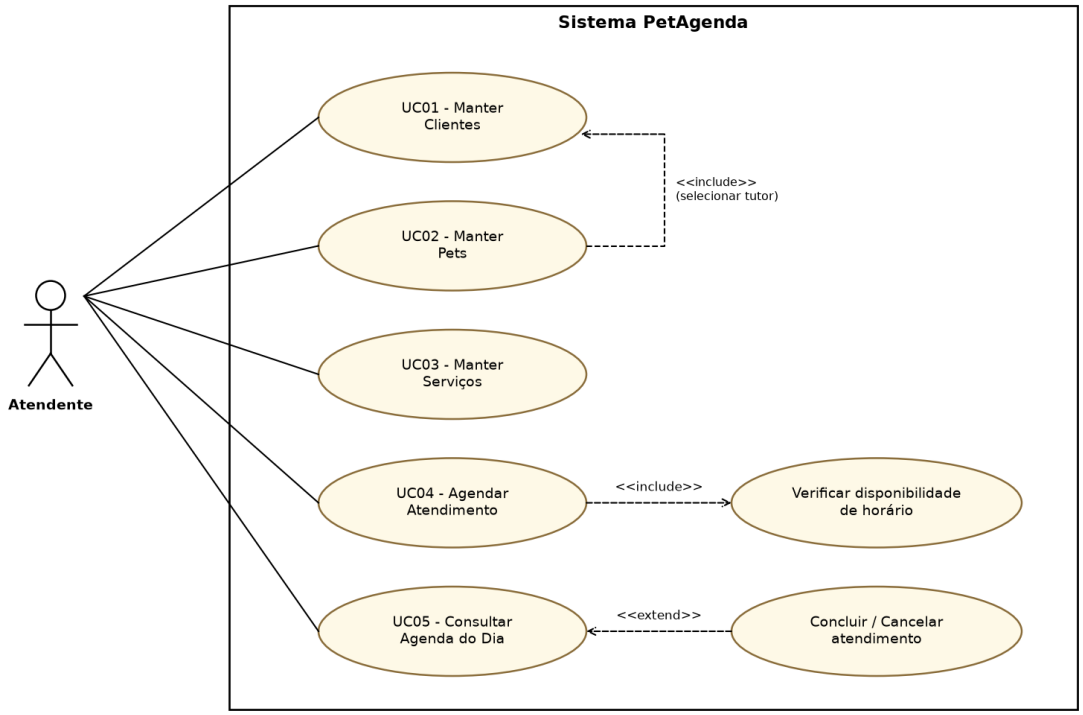
RNF04	Interface em português, com layout responsivo para telas pequenas.
RNF05	Consultas SQL parametrizadas (prepared statements) para evitar SQL Injection.
RNF06	Mensagens claras de sucesso e erro após cada operação.
RNF07	Código versionado no GitHub, com testes automatizados das principais regras.

Fonte: os autores (2026).

2.3 Diagrama de casos de uso

O sistema tem um ator, o Atendente, que é o funcionário que recebe os clientes e organiza a agenda. Foram definidos cinco casos de uso principais, mostrados na Figura 1.

Figura 1 - Diagrama de casos de uso



Fonte: os autores (2026).

Quadro 4 - Descrição dos casos de uso

Caso de uso	Ator	Descrição
UC01 - Manter Clientes	Atendente	Cadastra, consulta, altera e exclui os tutores dos pets.
UC02 - Manter Pets	Atendente	Cadastra, consulta, altera e exclui pets. Inclui a seleção do tutor.

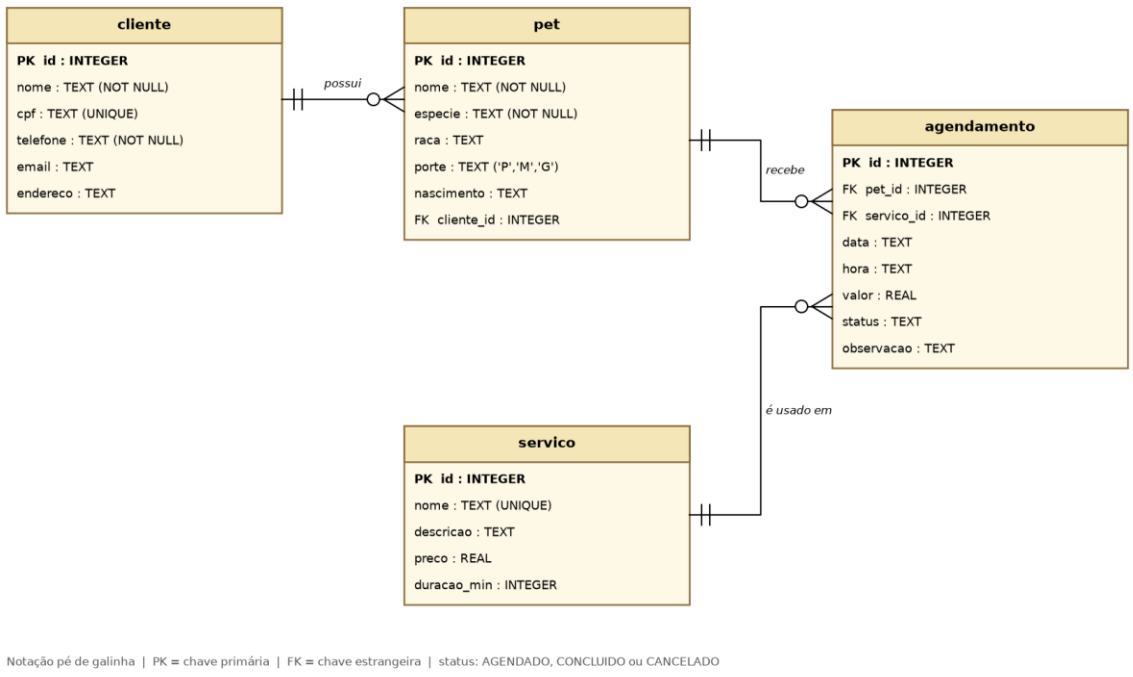
UC03 - Manter Serviços	Atendente	Mantém a tabela de serviços (banho, tosa, consulta etc.).
UC04 - Agendar Atendimento	Atendente	Marca um serviço para um pet. Inclui a verificação de disponibilidade do horário.
UC05 - Consultar Agenda do Dia	Atendente	Mostra os atendimentos do dia e os indicadores. Pode ser estendido por Concluir/Cancelar atendimento.

Fonte: os autores (2026).

2.4 Diagrama Entidade-Relacionamento (DER)

O banco possui quatro tabelas. Um cliente pode ter vários pets, e todo pet pertence a um cliente. Cada agendamento liga um pet a um serviço. O valor é copiado do serviço no momento do agendamento, para que uma mudança de preço depois não altere atendimentos antigos. A coluna status aceita apenas AGENDADO, CONCLUIDO ou CANCELADO (restrição CHECK).

Figura 2 - Diagrama Entidade-Relacionamento

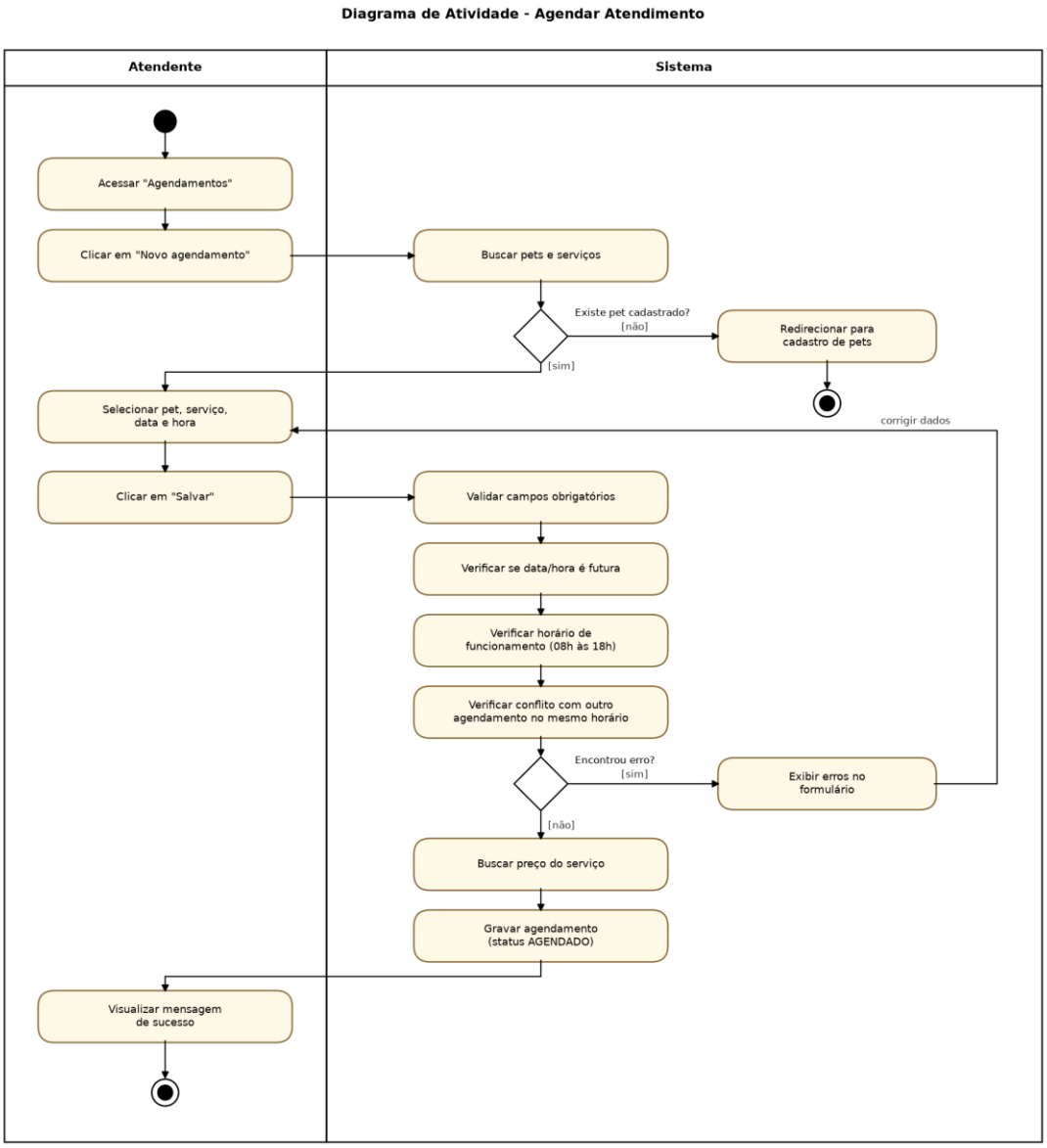


Fonte: os autores (2026).

2.5 Diagrama de atividade

Escolhemos representar o processo de agendar um atendimento, que é o fluxo com mais regras do sistema. Antes de gravar, o sistema confere se os campos foram preenchidos, se a data não é passada, se está dentro do horário de funcionamento e se o horário está livre. Se algo estiver errado, o atendente volta ao formulário com as mensagens de erro.

Figura 3 - Diagrama de atividade - Agendar atendimento

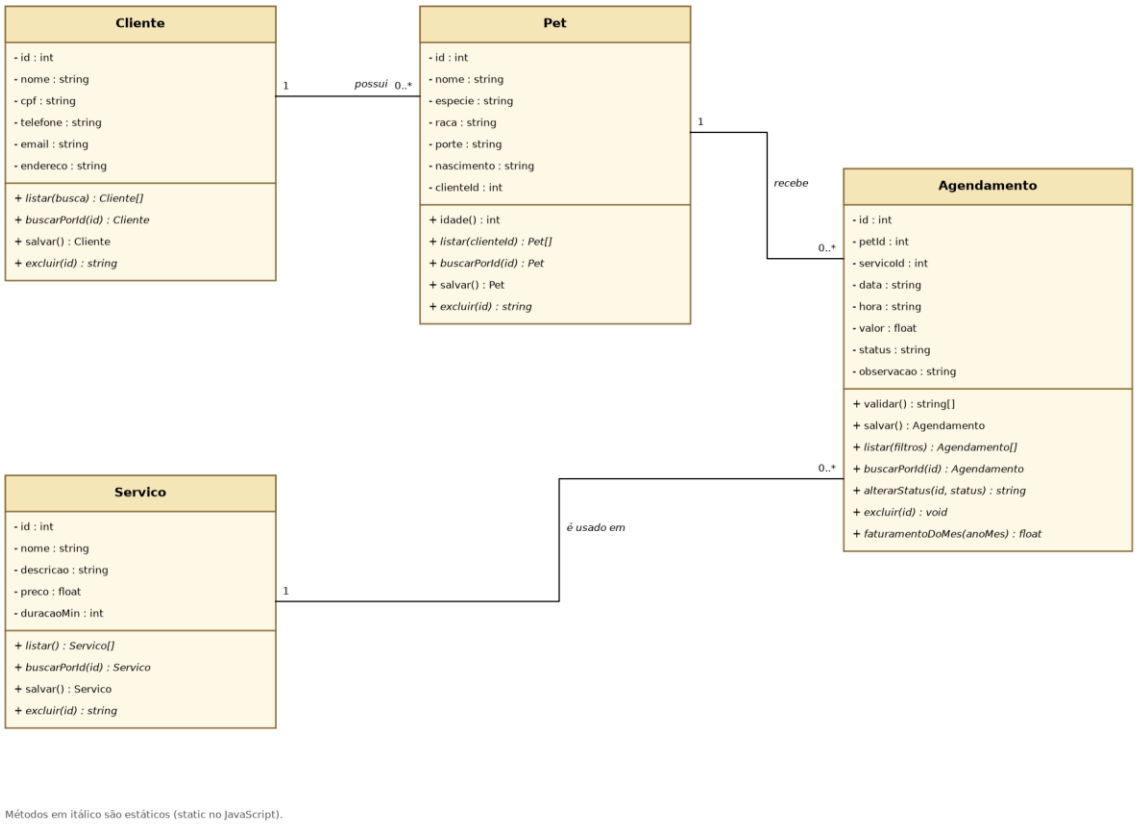


Fonte: os autores (2026).

2.6 Diagrama de classes

O diagrama de classes é focado nas entidades. Cada classe guarda os dados da entidade e tem os métodos de acesso ao banco. Os métodos de consulta e exclusão são estáticos, pois não dependem de um objeto já criado. A classe Agendamento concentra as regras de negócio no método validar().

Figura 4 - Diagrama de classes



Fonte: os autores (2026).

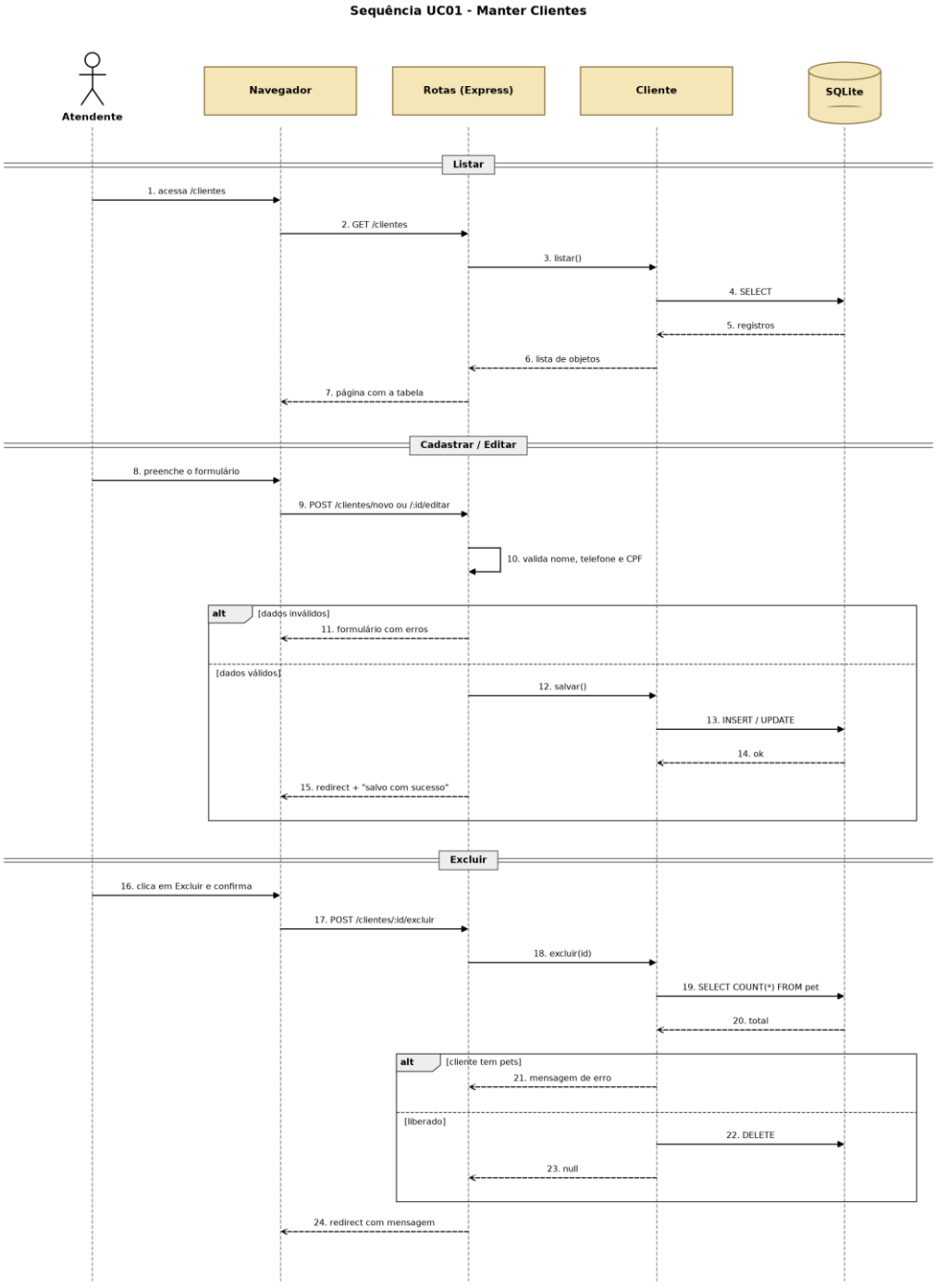
2.7 Diagramas de sequência

Foi feito um diagrama de sequência para cada caso de uso. Neles aparecem o atendente, o navegador, as rotas do Express, a classe da entidade e o banco SQLite.

2.7.1 UC01 - Manter Clientes

Na exclusão, o sistema primeiro conta quantos pets o cliente tem. Se tiver algum, a exclusão é bloqueada.

Figura 5 - Diagrama de sequência - UC01 Manter Clientes

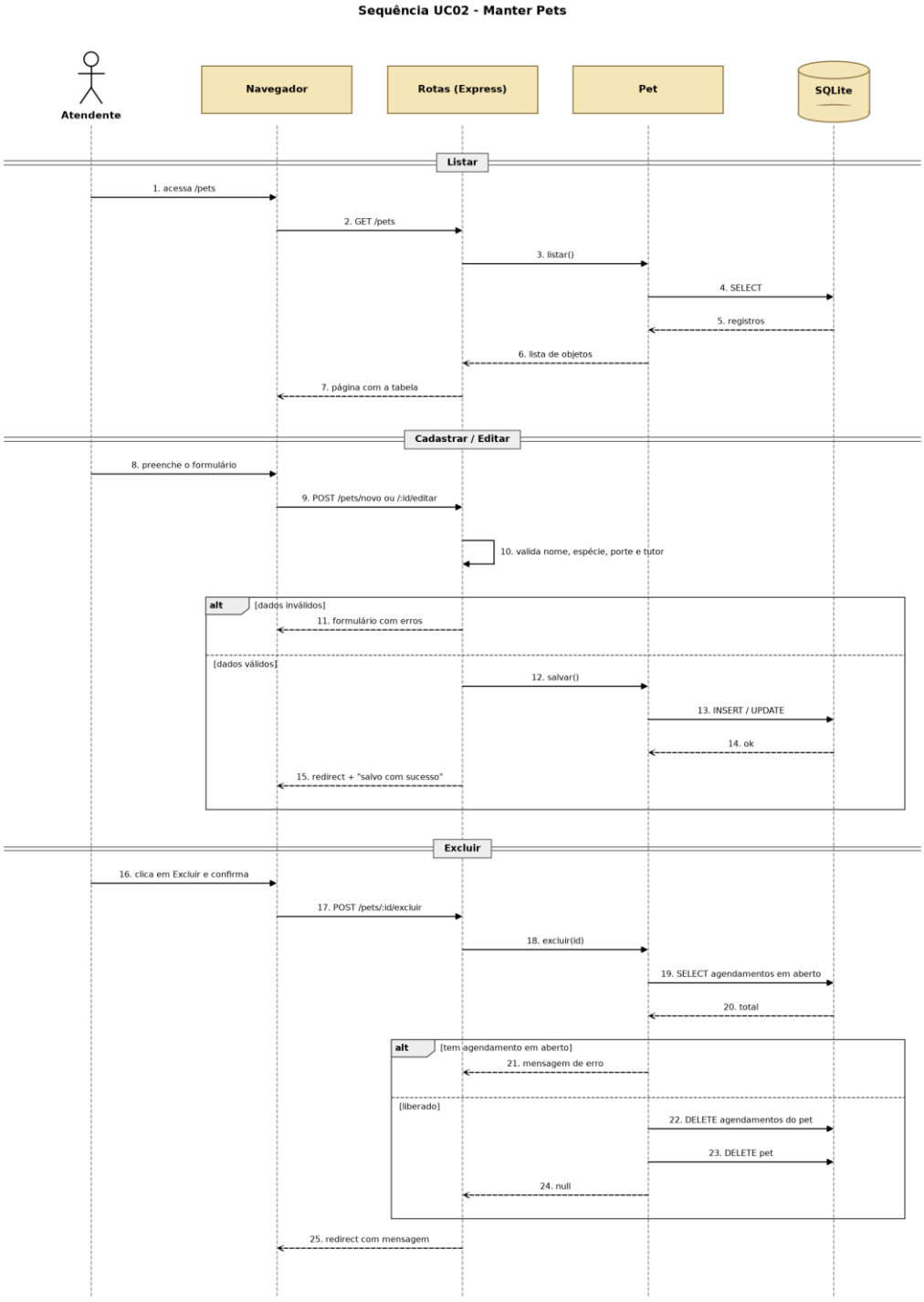


Fonte: os autores (2026).

2.7.2 UC02 - Manter Pets

Um pet só pode ser excluído se não tiver agendamento em aberto. Nesse caso o histórico de atendimentos dele também é apagado.

Figura 6 - Diagrama de sequência - UC02 Manter Pets

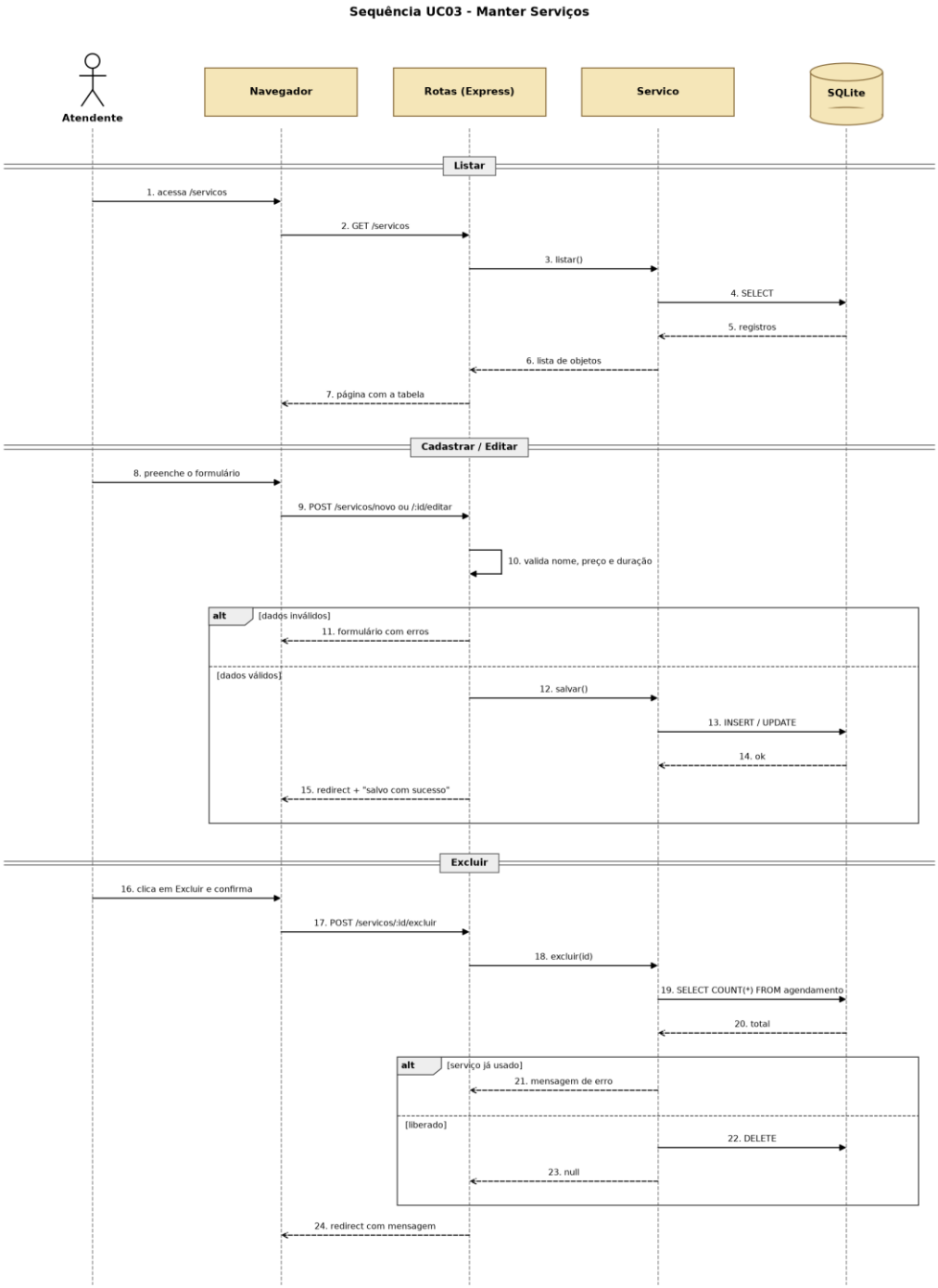


Fonte: os autores (2026).

2.7.3 UC03 - Manter Serviços

Serviços que já foram usados em algum agendamento não podem ser excluídos, para não perder o histórico.

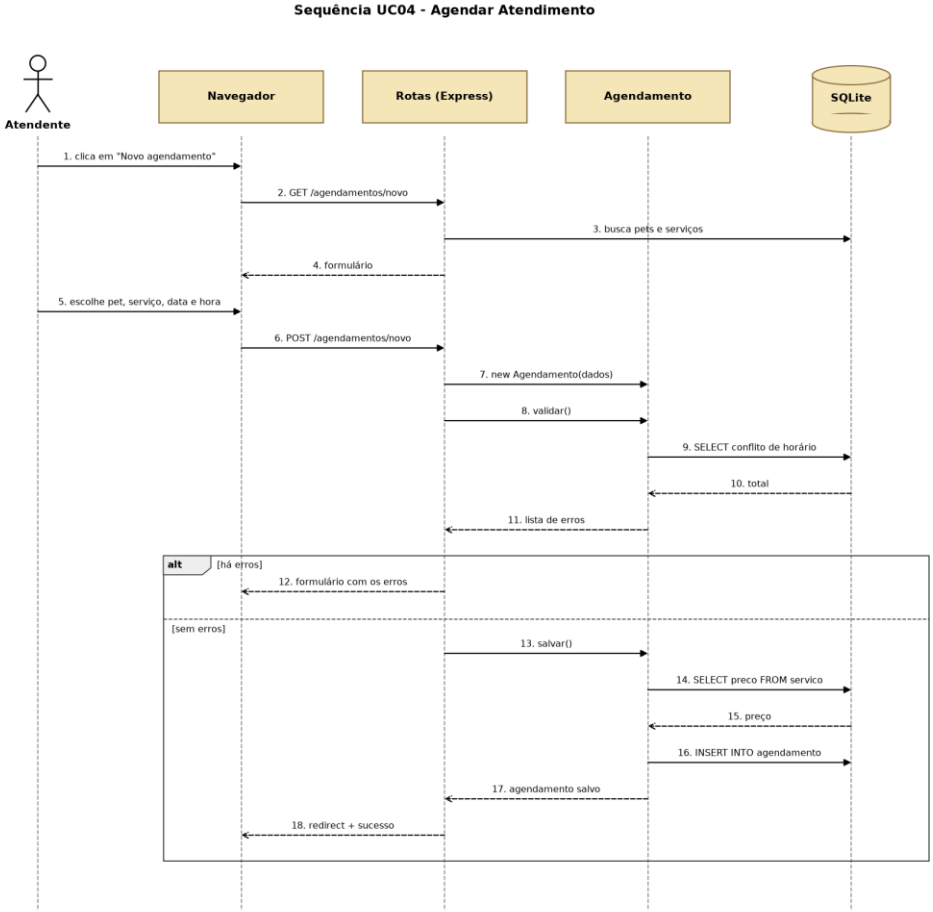
Figura 7 - Diagrama de sequência - UC03 Manter Serviços



Fonte: os autores (2026).

2.7.4 UC04 - Agendar Atendimento

Figura 8 - Diagrama de sequência - UC04 Agendar Atendimento

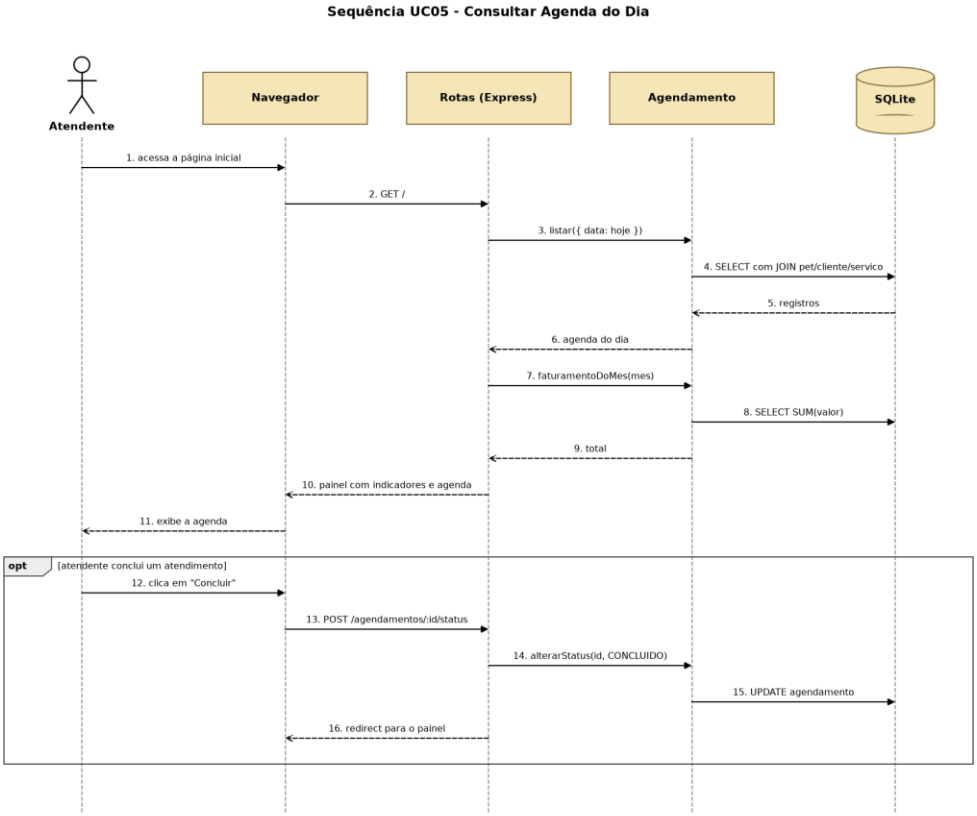


Fonte: os autores (2026).

2.7.5 UC05 - Consultar Agenda do Dia

O bloco opt mostra a extensão do caso de uso, quando o atendente conclui um atendimento direto pelo painel.

Figura 9 - Diagrama de sequência - UC05 Consultar Agenda do Dia



Fonte: os autores (2026).

3 CONCLUSÃO

A etapa de documentação mostrou que pensar no sistema antes de programar economiza bastante tempo. Durante o levantamento de requisitos percebemos regras que não tinham passado pela nossa cabeça no começo, como o que fazer quando o preço de um serviço muda, ou se um cliente com pets pode ser excluído.

O DER ajudou a decidir onde guardar cada informação, por exemplo copiar o valor do serviço para o agendamento. Os diagramas de sequência deixaram claro o caminho de uma requisição desde o navegador até o banco, o que serviu de guia para a implementação na Parte II.

Também aprendemos a dividir o trabalho em dupla usando o GitHub, cada um cuidando de uma parte e revisando a do outro.

REFERÊNCIAS

EJS. Embedded JavaScript templating. Disponível em: https://ejs.co/. Acesso em: nov. 2026.

EXPRESS. Express 4.x - API Reference. OpenJS Foundation. Disponível em: https://expressjs.com/. Acesso em: nov. 2026.

GUEDES, Gilleanes T. A. UML 2: uma abordagem prática. 3. ed. São Paulo: Novatec, 2018.

HEUSER, Carlos Alberto. Projeto de banco de dados. 6. ed. Porto Alegre: Bookman, 2009.

NODE.JS. SQLite | Node.js v24 Documentation. Disponível em: https://nodejs.org/api/sqlite.html. Acesso em: nov. 2026.

NODE.JS. Test runner | Node.js v24 Documentation. Disponível em: https://nodejs.org/api/test.html. Acesso em: nov. 2026.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson, 2019.

SQLITE. SQLite Documentation. Disponível em: https://www.sqlite.org/docs.html. Acesso em: nov. 2026.